Literature Survey: ML-Guided Logic Synthesis Recipe Selection

Shaan Chopra (240905140), Kenneth Rodrigues (240905112),
Darren Pais (240905093), and Aniket Verma (240905156)

School of Computer Science and Engineering
Manipal Institute of Technology

1 Survey of Prior Work

Mishchenko, Chatterjee, and Brayton (2006) addressed the reliance of traditional logic synthesis on trial-and-error and hand-tuned optimization scripts by proposing a scalable technology-independent flow to run before technology mapping. They represented combinational logic as And-Inverter Graphs (AIGs) and evaluated the flow on large industrial benchmarks. The flow alternated DAG-aware AIG rewriting, which reduced area by sharing common logic without increasing delay, with algebraic AIG balancing, which reduced delay without increasing area, and the authors implemented it in the public-domain tool ABC [1]. They reported that the methodology scaled to very large designs and ran several orders of magnitude faster than SIS and MVSIS, with comparable or better quality measured after mapping. The flow is limited by its fixed transformation sequence, which is applied to every circuit regardless of structure.

Hosny et al. (2020) addressed the exponentially large space of optimization sequences, where quality of results (QoR) depends on the order of transformations, by automating flow exploration with reinforcement learning. They used designs from the EPFL benchmark suite, loaded them into ABC, and mapped them to the ASAP7 library, measuring delay with ABC's built-in timer and choosing the delay constraint from an initial unoptimized run. The authors trained an Advantage Actor Critic (A2C) agent to minimize area subject to that timing constraint and compared it against the best published EPFL results, expert-crafted scripts, and a greedy heuristic [2]. They reported that the agent outperformed existing exploration methods and improved QoR by 13% on average. The method is limited by the synthesis-run cost of RL-based exploration.

Chowdhury et al. (2021) addressed the absence of standard datasets and prototypical learning tasks for ML-guided logic synthesis by releasing OpenABC-D. They synthesized 29 open-source hardware IPs collected from several public sources with 1500 random recipes of length 20, and preserved the starting, intermediate, and final AIGs, which produced about 870,000 AIGs labeled with node count, longest path, area, and delay. The authors trained a two-layer graph convolutional network (GCN) with global max and average pooling, together with a linear and 1D-convolution encoding of the recipe, to predict post-synthesis AIG node count under three splits: unseen recipes (1000 recipes for training, 500 for testing), unseen larger IPs (16 smaller IPs for training, 8 larger IPs for testing), and unseen IP–recipe pairs [3]. They reported baseline test error for each split, with the split in which both IPs and recipes had been observed during training giving the best performance. The baseline GCN is limited by weak generalization to

unseen designs.

Lau Neto et al. (2022) addressed the difficulty of building IP-specific synthesis flows, noting that existing ML techniques either needed large labeled datasets and long training or carried high integration overhead. They evaluated their framework, FlowTune, on AIG node count, CNF clause count for SAT, post-STA delay and area for standard-cell mapping, and 6-input LUT FPGA mapping, as well as on an end-to-end FPGA flow assessed after routing with OpenFPGA and VPR, using both AIGs and MIGs. The authors formulated flow exploration as a domain-specific, multi-stage multi-armed bandit that exploited knowledge of DAG-aware synthesis transformations, with a rapid start-up phase and a multi-stage exploration strategy, and integrated it with ABC, Yosys, VTR, LSOracle, and OpenFPGA [4]. They reported that FlowTune outperformed hand-crafted flows and ML-explored flows in QoR and ran orders of magnitude faster than ML-based approaches. The framework is limited by online exploration that requires fresh synthesis runs for every new circuit.

2 Comparative Summary

Author(s)	**Year**	**Model / Method Used**	**Key Contribution**	**Study Limitation**
Mishchenko, Chatterjee & Brayton	2006	DAG-aware AIG rewriting + algebraic balancing (in ABC)	Scalable technology-independent flow that improves post-mapping quality with far lower runtime than SIS and MVSIS	A fixed transformation sequence for every circuit is limiting
Hosny et al.	2020	Advantage Actor Critic (A2C) reinforcement learning	Autonomous area minimization under a timing constraint, with 13% average QoR improvement on EPFL	Synthesis-run cost of RL exploration is limiting
Chowdhury et al.	2021	Two-layer GCN with recipe encoding	OpenABC-D: about 870,000 labeled AIGs from 29 IPs and 1500 recipes, with QoR prediction baselines	Weak generalization to unseen designs is limiting
Lau Neto et al.	2022	Domain-specific multi-stage multi-armed bandit (FlowTune)	End-to-end flow exploration across AIG, CNF, standard-cell, and FPGA tasks, faster than ML-based exploration	Fresh synthesis runs for every new circuit are limiting

Table 1: Comparison of the surveyed logic-synthesis optimization and ML-guided exploration methods.

3 Positioning of the Present Study

The surveyed work either applies one fixed flow to all circuits (Mishchenko et al. 2006), searches for a flow separately for each design (Hosny et al. 2020; Lau Neto et al. 2022), or predicts QoR with graph neural networks over full AIGs (Chowdhury et al. 2021). The present study takes a smaller supervised route: it predicts normalized mapped area and delay from inexpensive

structural circuit features and an explicit recipe encoding, evaluates on held-out circuits, and compares the selected recipe against `resyn2`, random selection, and the best fixed recipe from the training circuits.

References

[1] Mishchenko, A., Chatterjee, S., Brayton, R.: DAG-Aware AIG Rewriting: A Fresh Look at Combinational Logic Synthesis. In: Proceedings of the 43rd ACM/IEEE Design Automation Conference (DAC), pp. 532–535 (2006).

[2] Hosny, A., Hashemi, S., Shalan, M., Reda, S.: DRiLLS: Deep Reinforcement Learning for Logic Synthesis. In: 25th Asia and South Pacific Design Automation Conference (ASP-DAC), pp. 581–586 (2020).

[3] Chowdhury, A.B., Tan, B., Karri, R., Garg, S.: OpenABC-D: A Large-Scale Dataset for Machine Learning Guided Integrated Circuit Synthesis. arXiv:2110.11292 (2021).

[4] Lau Neto, W., Li, Y., Gaillardon, P.-E., Yu, C.: FlowTune: End-to-End Automatic Logic Optimization Exploration via Domain-Specific Multi-Armed Bandit. IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems (2022).